

# 前端代码规范工具链搭建 SOP

(ESLint + Prettier + Stylelint + Husky + lint-staged)

适用场景：React / Vue / TypeScript 前端项目，团队协作代码规范统一。

## 一、整体思路

三个工具分工不同，不要让它们互相打架：

| 工具        | 职责                    | 不管什么             |
|-----------|-----------------------|------------------|
| ESLint    | JS/TS 代码质量、逻辑错误、最佳实践  | 不管代码格式（缩进/引号/分号） |
| Prettier  | 代码格式化（缩进、引号、换行、分号）    | 不管逻辑问题           |
| Stylelint | CSS/SCSS/Less 样式规范与格式 | 不管 JS            |

核心原则：Prettier 负责"好不好看"，ESLint/Stylelint 负责"对不对"，两者通过插件解除冲突。

## 二、前置条件

```
bash

node -v    # 建议 Node 16+
npm -v     # 或使用 pnpm/yarn
```

确保项目已有 `package.json`（没有则 `npm init -y`）。

## 三、Step 1: 安装 Prettier

```
bash

npm install --save-dev prettier
```

根目录新建 `.prettierrc.json`：

json

```
{
  "printWidth": 100,
  "tabWidth": 2,
  "useTabs": false,
  "semi": true,
  "singleQuote": true,
  "trailingComma": "all",
  "bracketSpacing": true,
  "arrowParens": "always",
  "endOfLine": "lf"
}
```

新建 `.prettierrignore`:

```
node_modules
dist
build
coverage
*.min.js
package-lock.json
pnpm-lock.yaml
```

## 四、Step 2: 安装 ESLint 并接入 Prettier

### 4.1 安装核心依赖

bash

```
npm install --save-dev eslint eslint-config-prettier eslint-plugin-prettier
```

- `eslint-config-prettier`: 关闭 ESLint 中与 Prettier 冲突的格式化规则
- `eslint-plugin-prettier`: 把 Prettier 的检查结果作为 ESLint 报错显示

### 4.2 按技术栈追加依赖

React 项目:

```
bash
```

```
npm install --save-dev eslint-plugin-react eslint-plugin-react-hooks
```

#### TypeScript 项目：

```
bash
```

```
npm install --save-dev @typescript-eslint/parser @typescript-eslint/eslint-plugin
```

#### Vue 项目：

```
bash
```

```
npm install --save-dev eslint-plugin-vue
```

#### 4.3 生成配置文件 `.eslintrc.cjs`（以 React + TS 为例）

```
js
```

```
module.exports = {
  root: true,
  env: {
    browser: true,
    es2021: true,
    node: true,
  },
  parser: '@typescript-eslint/parser',
  parserOptions: {
    ecmaVersion: 'latest',
    sourceType: 'module',
    ecmaFeatures: { jsx: true },
  },
  extends: [
    'eslint:recommended',
    'plugin:@typescript-eslint/recommended',
    'plugin:react/recommended',
    'plugin:react-hooks/recommended',
    'plugin:prettier/recommended', // 必须放最后，覆盖冲突规则
  ],
  plugins: ['@typescript-eslint', 'react', 'react-hooks'],
  settings: {
    react: { version: 'detect' },
  },
  rules: {
    'prettier/prettier': 'warn',
    'react/react-in-jsx-scope': 'off', // React 17+ 新 JSX 转换不需要
    '@typescript-eslint/no-unused-vars': 'warn',
    '@typescript-eslint/explicit-module-boundary-types': 'off',
  },
};
```

新建 `.eslintignore`:

```
node_modules
dist
build
*.config.js
```

## 五、Step 3: 安装 Stylelint 并接入 Prettier

### 5.1 安装

```
bash

npm install --save-dev stylelint stylelint-config-standard stylelint-config-prettier
```

如项目用 SCSS / Less:

```
bash

npm install --save-dev stylelint-config-standard-scss
```

如用 CSS-in-JS (styled-components):

```
bash

npm install --save-dev stylelint-config-styled-components
```

### 5.2 配置文件 `.stylelintrc.json`

```
json
```

```
{
  "extends": [
    "stylelint-config-standard",
    "stylelint-config-standard-scss",
    "stylelint-config-prettier"
  ],
  "rules": {
    "selector-class-pattern": null,
    "no-descending-specificity": null,
    "scss/at-import-partial-extension": null
  },
  "ignoreFiles": ["**/node_modules/**", "**/dist/**", "**/build/**"]
}
```

`stylelint-config-prettier` 必须放在 extends 最后，作用同 eslint-config-prettier：关闭冲突的格式规则。

## 六、Step 4：统一配置 package.json 脚本

```
json
{
  "scripts": {
    "lint:js": "eslint \"src/**/*.js,jsx,ts,tsx\" --fix",
    "lint:style": "stylelint \"src/**/*.css,scss,less\" --fix",
    "format": "prettier --write \"src/**/*.js,jsx,ts,tsx,css,scss,less,json,md\"",
    "lint": "npm run lint:js && npm run lint:style"
  }
}
```

验证：

```
bash
npm run lint
npm run format
```

## 七、Step 5: 接入 Git Hooks, 强制提交前检查 (推荐必做)

### 7.1 安装 husky + lint-staged

```
bash

npm install --save-dev husky lint-staged
npx husky init          # husky v9+
```

### 7.2 配置 `package.json`

```
json

{
  "lint-staged": {
    "*. {js,jsx,ts,tsx}": ["eslint --fix", "prettier --write"],
    "*. {css,scss,less}": ["stylelint --fix", "prettier --write"],
    "*. {json,md}": ["prettier --write"]
  }
}
```

### 7.3 编辑 `.husky/pre-commit`

```
bash

npx lint-staged
```

这样每次 `git commit` 前会自动格式化+检查暂存区文件, 不合规则直接阻止提交。

---

## 八、Step 6: 编辑器集成 (VS Code)

`.vscode/settings.json`:

```
json
```

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit",
    "source.fixAll.stylelint": "explicit"
  },
  "eslint.validate": ["javascript", "javascriptreact", "typescript", "typescriptreact"],
  "stylelint.validate": ["css", "scss", "less"]
}
```

要求团队安装 VS Code 插件：

- ESLint
- Prettier - Code formatter
- Stylelint

---

## 九、Step 7：CI 集成（可选但建议）

在 GitHub Actions / GitLab CI 中增加检查步骤，防止本地漏检的代码合入主分支：

```
yaml
# .github/workflows/lint.yml
name: Lint
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - run: npm ci
      - run: npm run lint
```

十、常见坑位排查表

| 现象                                 | 原因                                                         | 解决                                                   |
|------------------------------------|------------------------------------------------------------|------------------------------------------------------|
| ESLint 和 Prettier 报错冲突（如缩进/引号反复横跳） | 没装 <code>eslint-config-prettier</code> 或没放在 extends 最后     | 检查 extends 顺序                                        |
| Stylelint 对 SCSS 语法报语法错误           | 用了 <code>stylelint-config-standard</code> 但没装 scss 扩展      | 补装 <code>stylelint-config-standard-scss</code>       |
| VS Code 保存不自动修复                    | 未设置 <code>codeActionsOnSave</code> 或未安装对应插件                | 检查 <code>.vscode/settings.json</code>                |
| commit 时 husky 不生效                 | Node 版本/husky 版本不一致，或 <code>.husky/pre-commit</code> 无执行权限 | <code>chmod +x .husky/pre-commit</code>              |
| 老项目存量代码全部报错                        | 规则一次性铺开导致海量 warning                                        | 先用 <code>--fix</code> 批量修复一轮，再逐步收紧规则等级（warn → error） |

十一、落地建议（团队推广顺序）

1. 先只跑 `format`（Prettier）全量格式化一次，统一历史代码风格，单独提交一次 commit
2. 再引入 ESLint/Stylelint，规则先设为 `warn`，观察一到两周
3. 稳定后逐步把关键规则升级为 `error`
4. 最后接入 husky + lint-staged 强制卡口
5. CI 层再加一道保险，防止绕过本地 hook（如 `--no-verify`）

如需要，我可以针对你项目的具体技术栈（Vue3 / React / 是否用 TS / Monorepo 等）生成一套可直接落地的配置文件包。